UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
Ingeniería de Sistemas

GUÍA PRÁCTICA SEMANA 07 — INFORME

Proyecto: TechStore — SPA de tienda de tecnología

Asignatura: Desarrollo de Aplicaciones Web (IS093A)

Unidad I: Desarrollo Web Frontend

Estudiante: Jackeline Vivanco Solano

Docente: Aguilar Coronación Miguel Elias

Código: 2022100549I

Huancayo, 14 de octubre de 2026

Repositorio: https://github.com/Jackeline-VS/techstore-react-hooks

Desarrollo de la práctica

Paso 1: Ciclo de render, StrictMode y Profiler

Se envolvió la aplicación en <StrictMode> en main.jsx. Se creó la vista Lab con un contador que ejecuta tres setState en un mismo clic y un componente hijo que recibe una prop fija. El análisis se hizo con el Profiler y no con console.log, ya que StrictMode ejecuta dos veces los componentes en desarrollo y los logs no reflejan el rendimiento real.

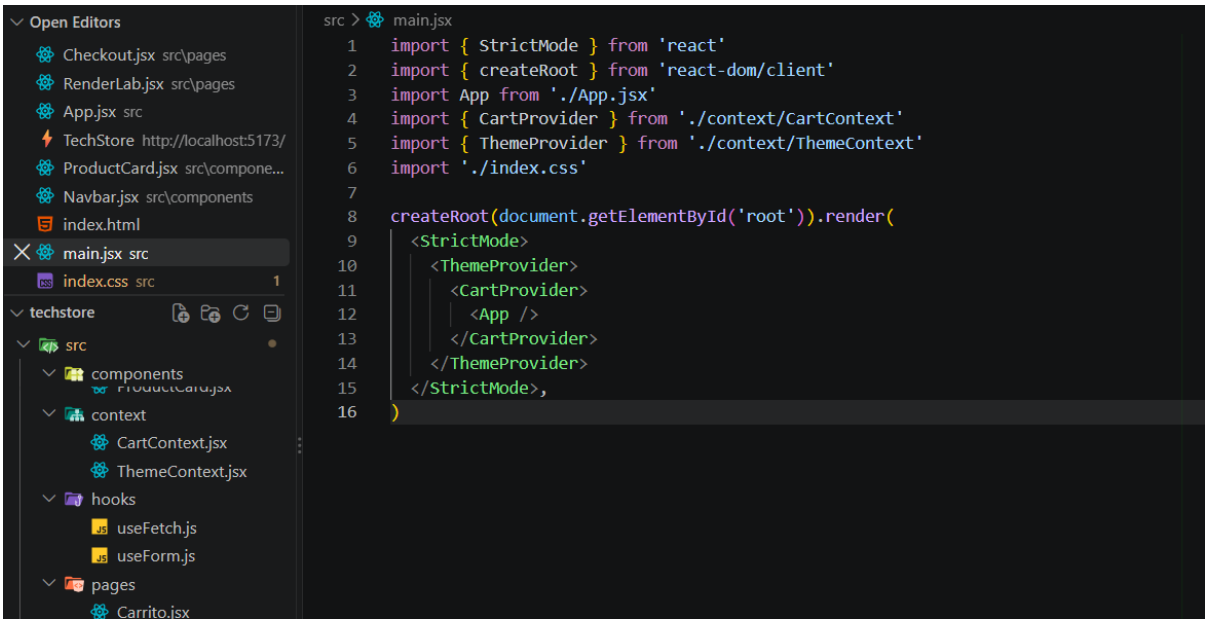


Figura 1. main.jsx: la app está envuelta en StrictMode y en los Providers de contexto.

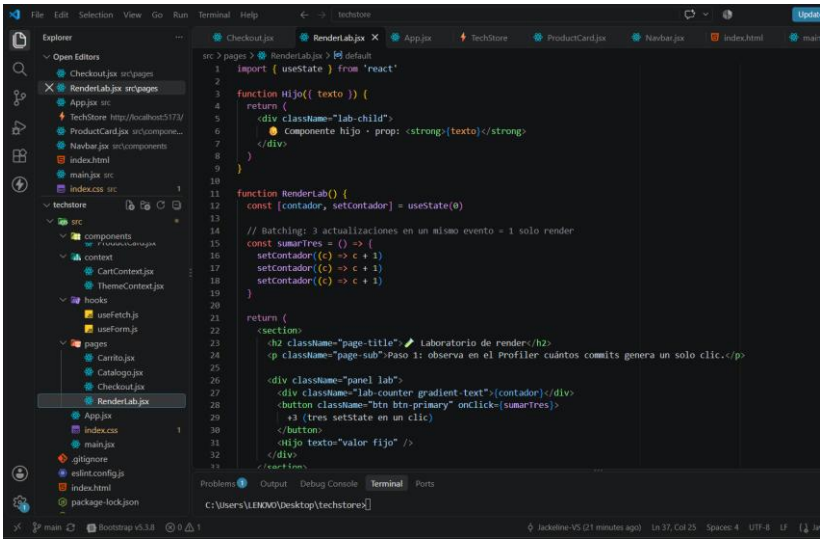


Figura 2. RenderLab.jsx: tres actualizaciones de estado dentro del mismo evento (batching).

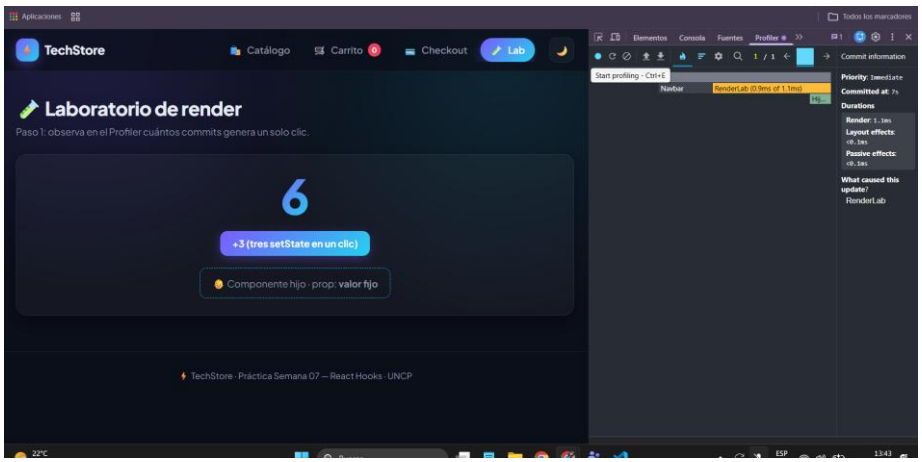


Figura 3. Profiler: un solo commit (1/1) a pesar de los tres setState. Navbar no se re-renderiza (gris).

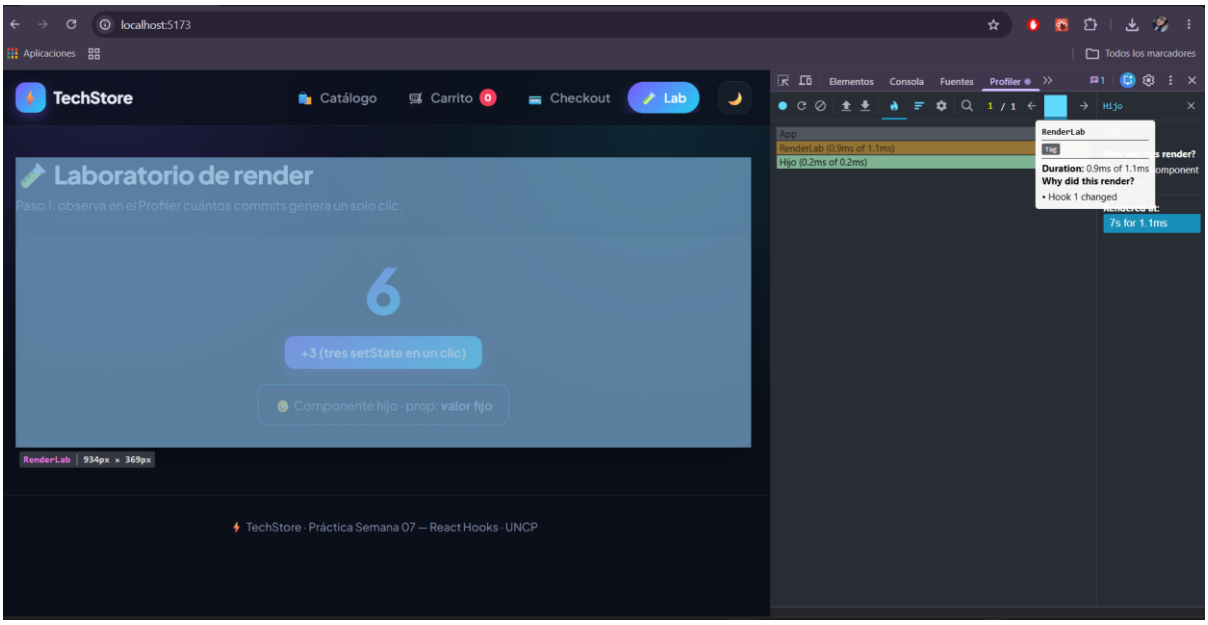


Figura 4. Profiler: RenderLab se renderiza por "Hook 1 changed" (useState) e Hijo porque su padre se renderizó.

Conclusión: React agrupa las actualizaciones de un mismo evento (batching). Un componente se re-renderiza cuando cambia su estado, cuando se renderiza su padre o cuando cambia un contexto que consume.

Paso 2: useEffect con async/await y AbortController

Se migró la petición fetch de .then() a async/await dentro de useEffect. Como useEffect no puede ser async, se define una función async interna. Se añadió un AbortController y la función de limpieza return () => controller.abort(), que cancela la petición si el componente se desmonta y evita memory leaks.

¿Por qué [ ] no significa "se ejecuta una vez"? El array vacío indica que el efecto no depende de valores reactivos. En desarrollo, StrictMode monta, desmonta y vuelve a montar el componente para comprobar que la limpieza funciona; por eso el efecto se ejecuta dos veces y la primera petición queda cancelada.

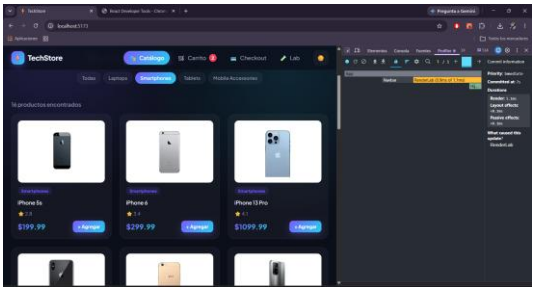


Figura 5. Catálogo cargado con productos reales de la API, filtrado por la categoría Smartphones.

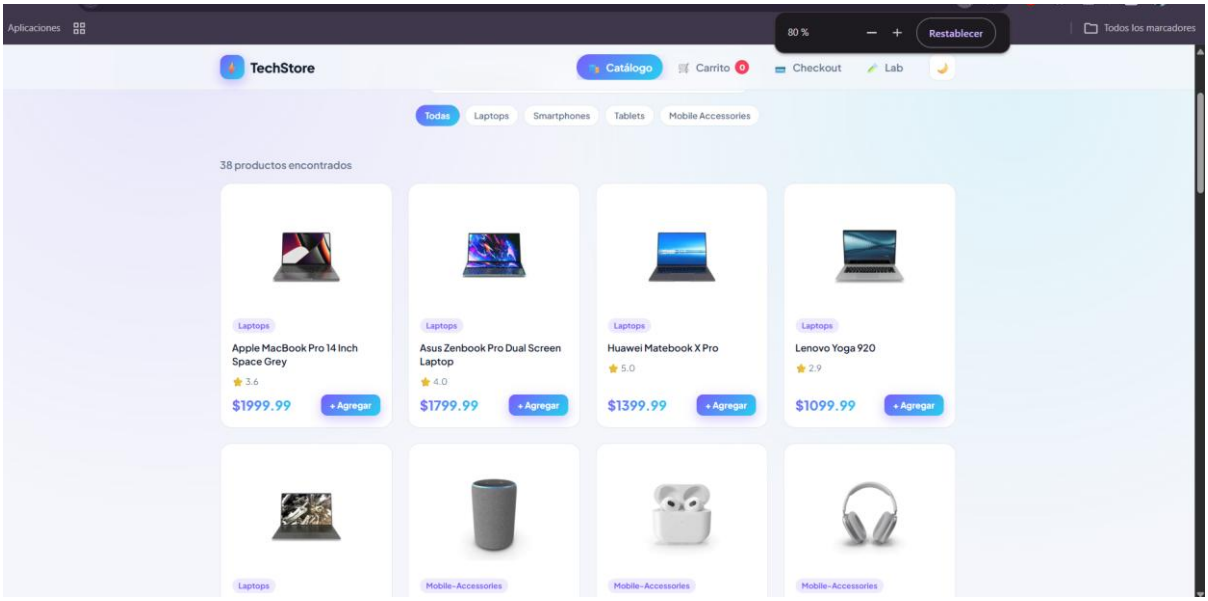


Figura 6. Catálogo completo (38 productos) con tarjetas de imagen, categoría, rating y precio.

Paso 3: useContext (tema y carrito)

Se crearon dos contextos separados por dominio, como recomienda la guía: ThemeContext (tema claro/oscuro) y CartContext (productos del carrito). Ambos se proveen en main.jsx y se consumen con los hooks useTheme() y useCart() desde Navbar, ProductCard, Carrito y Checkout, sin prop drilling.

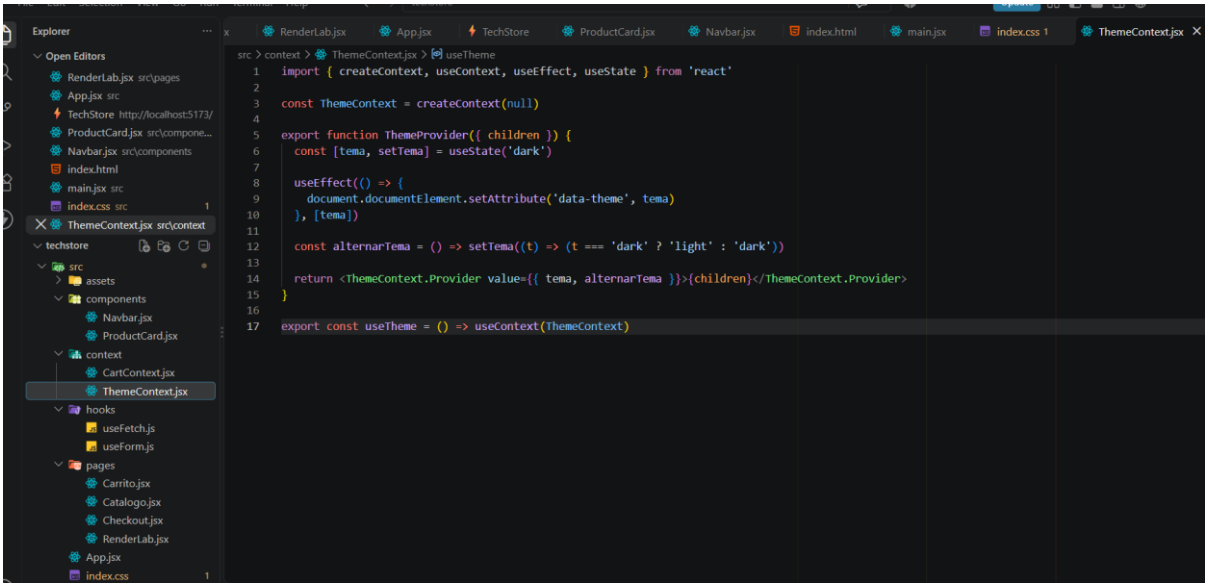


Figura 7. ThemeContext.jsx: Provider con useState y useEffect que aplica data-theme al documento.

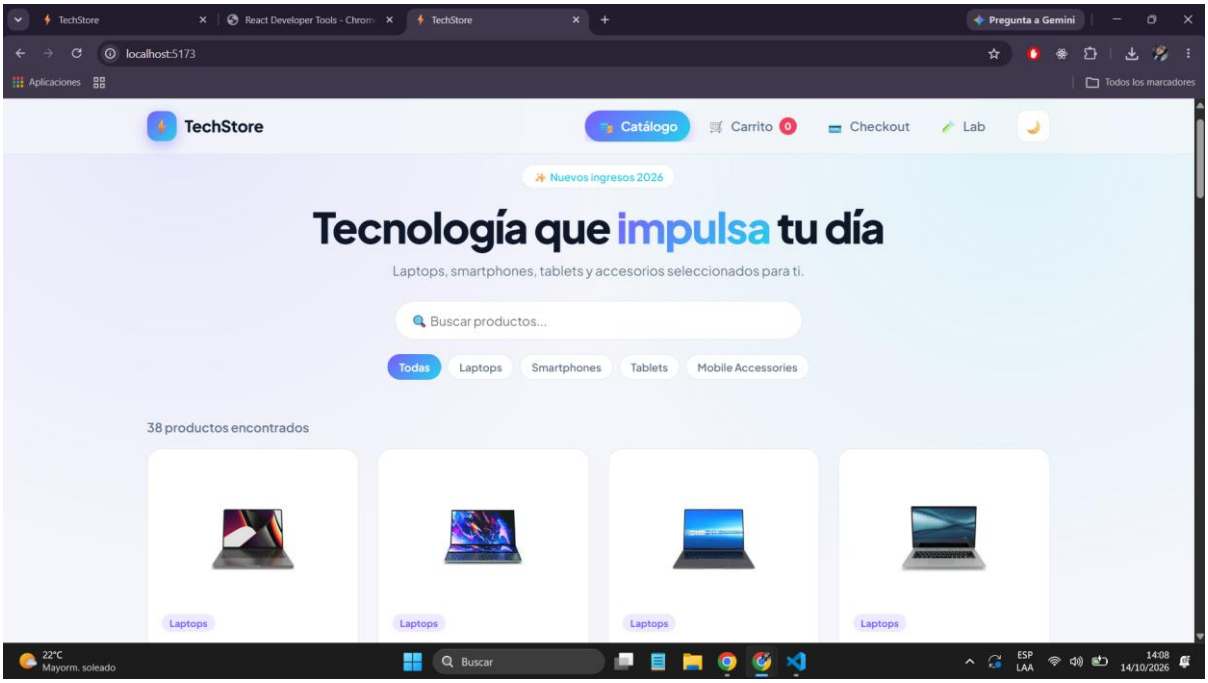


Figura 8. Tienda en tema claro, cambiado desde el botón del Navbar mediante useContext.

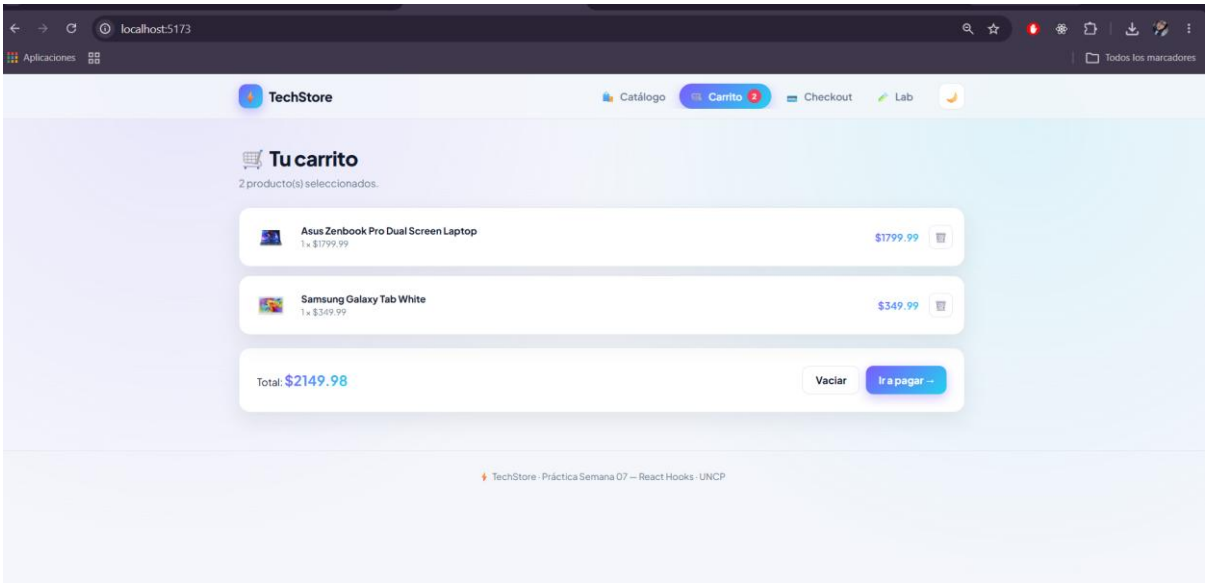


Figura 9. Carrito compartido por contexto: el contador del Navbar y el total se actualizan en toda la app.

Paso 4: useMemo, useCallback, React.memo y useRef

- **useMemo:** el filtro de productos (búsqueda + categoría) solo se recalcula cuando cambian productos, busqueda o categoria.
- **useCallback:** las funciones agregar, quitar y vaciar de CartContext mantienen la misma referencia entre renders.
- **React.memo:** ProductCard no se re-renderiza si sus props no cambian.
- **useRef:** da foco automático al buscador y al campo Nombre del Checkout sin provocar re-renders.

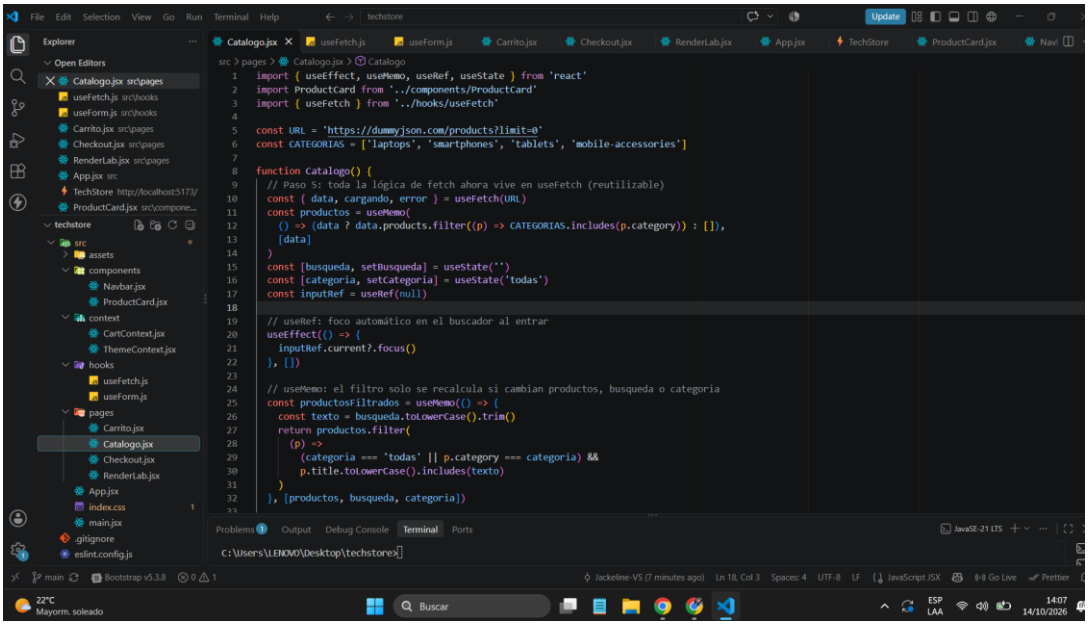


Figura 10. Catalogo.jsx: useRef para el foco y useMemo para el filtro de productos.

Paso 5: Hooks personalizados useFetch y useForm

Se extrajo la lógica repetitiva a hooks propios. useFetch encapsula la petición, los estados data/cargando/error y la limpieza con AbortController, y se reutiliza en el Catálogo. useForm maneja valores, cambios, validación y reset del formulario de Checkout. Se respetaron las reglas de Hooks: no se llaman dentro de loops, condiciones ni funciones anidadas, y el return condicional del Checkout va después de todos los hooks.

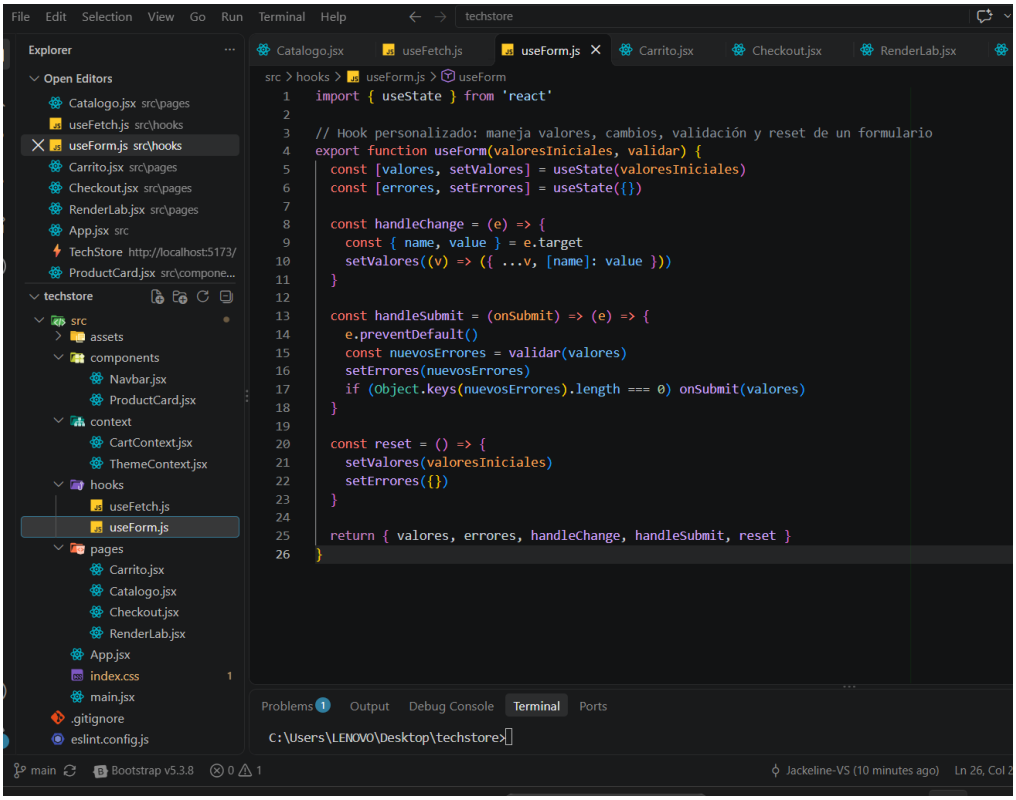


Figura 11. useForm.js: hook personalizado para valores, errores, handleChange, handleSubmit y reset.

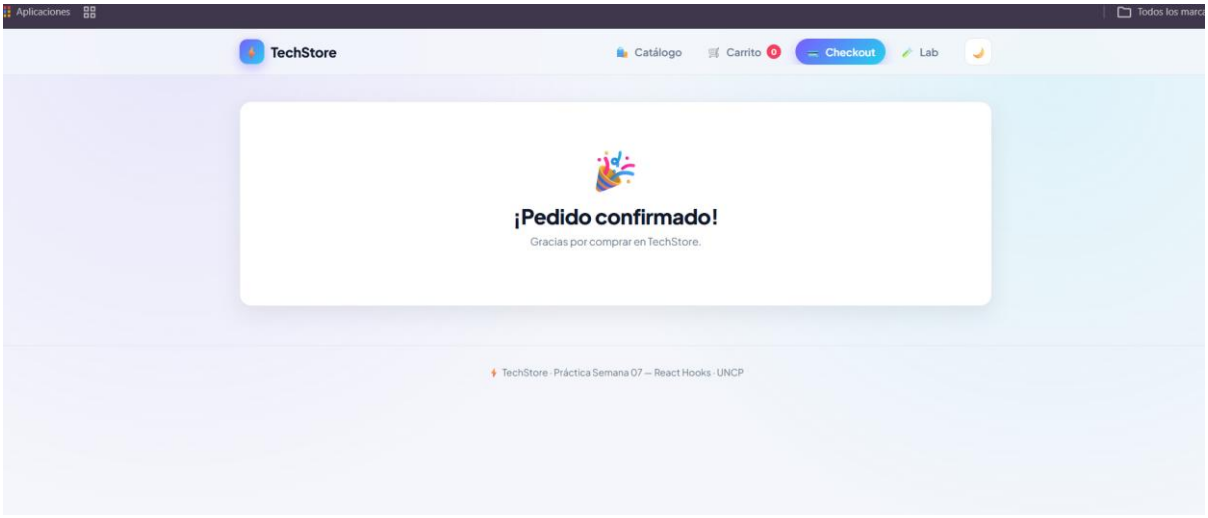


Figura 12. Checkout: pedido confirmado tras validar el formulario con useForm; el carrito se vacía.

Enlace del repositorio

https://github.com/Jackeline-VS/techstore-react-hooks